

如何使用LR实现函数的调用

调用函数Arm汇编demo

让deepseek写一个demo

```
.text
.globl _start

_start:
    mov r0, #1
    mov r1, #2
    bl add_func # bl在转时，会把下一行代码的地址存入LR
                # 转跳指令还有b、bx、blx
                # b跳完就再也不回来了
                # bx跳完也不回来，同时可以指定指令集
                # blx即写LR，又指定指令集

    mov r7, #1
    svc #0

add_func:
    add r0, r0, r1
    bx lr
```

bl指令可以实现函数的调用，从内存的角度看：

```
# 调用函数之前
    | bx lr          |
    | add r0, r0, r1 | add_func
    | .....         |
    | mov r7, #1     |
    | bl add_func    |
PC-8 ---->| mov r1, #2 | # 由于Arm流水线机制，正在执行的代码应当是是PC-8
           | mov r0, #1 |
```

```

# 调用函数中
        | bx lr          |
        | add r0, r0, r1 |<---| # PC-8 跳转到这里
        | .....         |
LR  ---->| mov r7, #1      |      | # LR记录返回地址
        | bl add_func    |----|
        | mov r1, #2      |
        | mov r0, #1      |

```

```

# 调用函数后
|-----| bx lr          |
|        | add r0, r0, r1 | add_func
|        | .....         |
|->PC-8 ->| mov r7, #1      | # LR记录的地址
        | bl add_func    |
        | mov r1, #2      |
        | mov r0, #1      |

```

注意：程序代码只会被放在堆里，栈里永远不会有程序代码

多层调用

如果只依赖LR，那只能进行1层函数调用，如果2层函数调用呢？

```

.text
.globl _start

_start:
    mov r0, #1
    bl func1
    svc #0

func1:
    push {lr} # 将lr的值压入栈中
    bl func2
    pop {pc} # 将func1对应的lr弹出，赋给PC

```

